Week 6 Assignment

Spark Architecture, Lazy Evaluation, Data Processing and File Format Optimization using Apache Spark

Submitted By: Aditya Kumar
Internship: Celebal Technologies – Data Engineering

Objective

The objective of this assignment is to understand the architecture of Apache Spark and its execution model while performing efficient data processing using Spark Data Frames. The assignment covers Spark Architecture, Lazy Evaluation, DAG (Lineage Graph), schema handling, filtering, transformations, actions, file format optimization (CSV and Parquet), null handling, and building an end-to-end data processing pipeline.

Q1: Explain the roles of the Driver, Cluster Manager, and Executor in a Spark application.

Answer -

Driver -

The Driver is the main process of a Spark application. It is responsible for creating the SparkSession, converting user code into execution plans (DAG), scheduling tasks, and coordinating the overall execution. It also collects the final results from the executors.

Cluster Manager -

The Cluster Manager is responsible for managing the resources of the cluster. It allocates CPU cores and memory to Spark applications and launches executors on worker nodes. Spark can work with different cluster managers such as Standalone, YARN, Mesos, and Kubernetes.

Executor -

Executors are worker processes that run on the worker nodes. They execute the tasks assigned by the Driver, perform computations, store intermediate data in memory or disk, and return the results back to the Driver.

Working Flow

1. The Driver receives the Spark application.
2. The Cluster Manager allocates the required resources.
3. Executors are launched on worker nodes.
4. Executors process the assigned tasks.
5. Results are sent back to the Driver.

Q2: How does Spark's Lazy Evaluation strategy improve performance when chain-processing large datasets?

Answer -

Lazy Evaluation -

Lazy Evaluation is a core optimization feature of Apache Spark. When transformations such as filter(), select(), or withColumn() are applied, Spark does not execute them immediately. Instead, it records these operations in a **Directed Acyclic Graph (DAG)**, also known as the **Lineage Graph**. Execution begins only when an **Action** such as show(), count(), collect(), or write() is called.

How It Improves Performance

- Spark combines multiple transformations into a single optimized execution plan.
- It avoids unnecessary intermediate computations.
- It minimizes disk I/O by processing data efficiently in memory.
- It optimizes task execution using the DAG Scheduler.
- It executes only the operations required to produce the final result.

Example

```
df.filter(df["price"] > 100) \
  .select("product_id", "price") \
  .show()
```

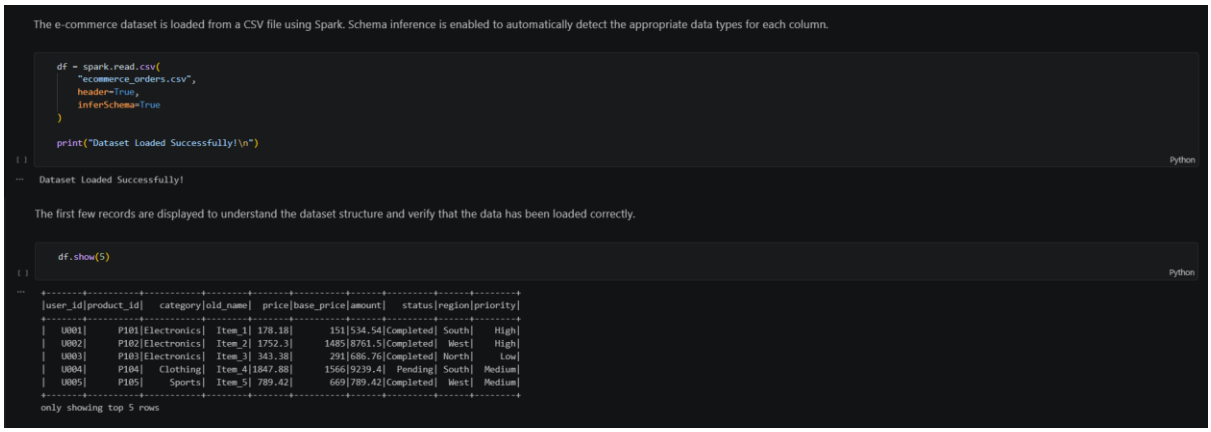
In this example, the filter() and select() transformations are **not executed immediately**. Spark records them in the DAG and performs both operations together only when show() is called.

Q3: Write a Spark command to read a CSV file located at "data/source.csv", ensuring the first row is treated as a header and inferSchema is enabled.

Answer -

The following Spark command reads a CSV file while treating the first row as the header and automatically inferring the data types of each column using inferSchema=True.

Screenshot:



Explanation

- spark.read.csv() reads the CSV file.
- header=True treats the first row as column names.
- inferSchema=True automatically detects the appropriate data type for each column.
- show(5) displays the first five records of the DataFrame.

Q4: Why is Parquet generally preferred over CSV for analytical workloads in Spark?

Answer -

Parquet is a **columnar storage format**, whereas CSV is a plain text row-based format. For analytical workloads, Spark prefers Parquet because it provides better performance, efficient storage, and optimized query execution.

Advantages of Parquet over CSV

- **Columnar Storage:** Only the required columns are read during query execution, reducing I/O operations.
- **Compression:** Parquet supports efficient compression techniques, reducing storage space.
- **Schema Preservation:** Data types are stored within the file, eliminating the need for schema inference every time the file is read.
- **Predicate Pushdown:** Spark reads only the necessary data blocks, improving query performance.
- **Faster Query Execution:** Reading fewer columns and compressed data significantly improves analytical performance.

Comparison

CSV	Parquet
Row-based storage	Column-based storage
Larger file size	Smaller compressed files
No schema information	Schema stored with data
Slower analytical queries	Faster analytical queries
Reads entire rows	Reads only required columns

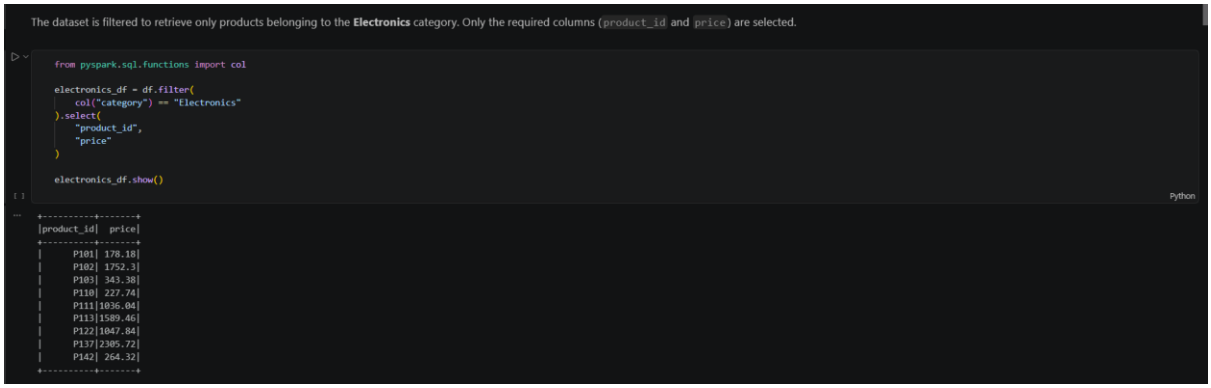
Therefore, Parquet is the preferred storage format for large-scale analytical workloads in Apache Spark.

Q5: Load a CSV dataset into a Spark DataFrame and display only the product_id and price columns where the category is Electronics.

Answer -

The following Spark code filters the dataset to retrieve only the products belonging to the **Electronics** category and displays only the product_id and price columns.

Screenshot:



Explanation

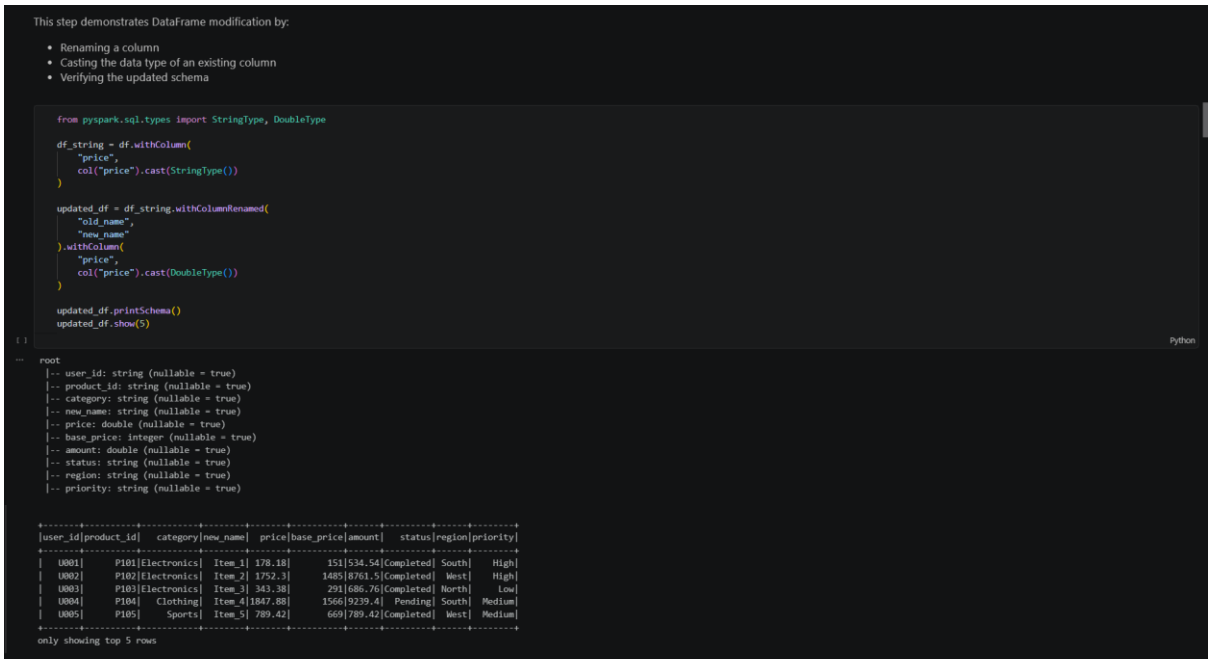
- filter() is used to retrieve only rows where the category is **Electronics**.
- select() displays only the required columns.
- show() displays the filtered DataFrame.

Q6: Rename the column old_name to new_name and cast the price column to DoubleType.

Answer -

Spark provides withColumnRenamed() to rename existing columns and cast() to convert data types. In this example, the column old_name is renamed to new_name, and the price column is cast to DoubleType.

Screenshot:



Explanation

- withColumnRenamed() changes the column name from old_name to new_name.
- cast(DoubleType()) converts the price column to the DoubleType data type.
- printSchema() verifies the updated schema.
- show(5) displays the modified DataFrame.

Q7: What is the Lineage Graph in Spark, and how does it help in fault tolerance?

Answer -

Lineage Graph

The Lineage Graph, also known as the **Directed Acyclic Graph (DAG)**, is a record of all the transformations applied to a Spark DataFrame or RDD. Instead of storing multiple copies of intermediate data, Spark records the sequence of operations performed on the data.

Role in Fault Tolerance

Spark uses the Lineage Graph to recover lost data automatically. If a partition is lost due to executor failure, Spark does not reload the entire dataset. Instead, it recomputes only the missing partition by re-executing the transformations recorded in the Lineage Graph.

Advantages

- Enables automatic fault recovery.
- Eliminates the need for data replication.
- Recomputes only the lost partitions.
- Improves reliability while reducing storage overhead.

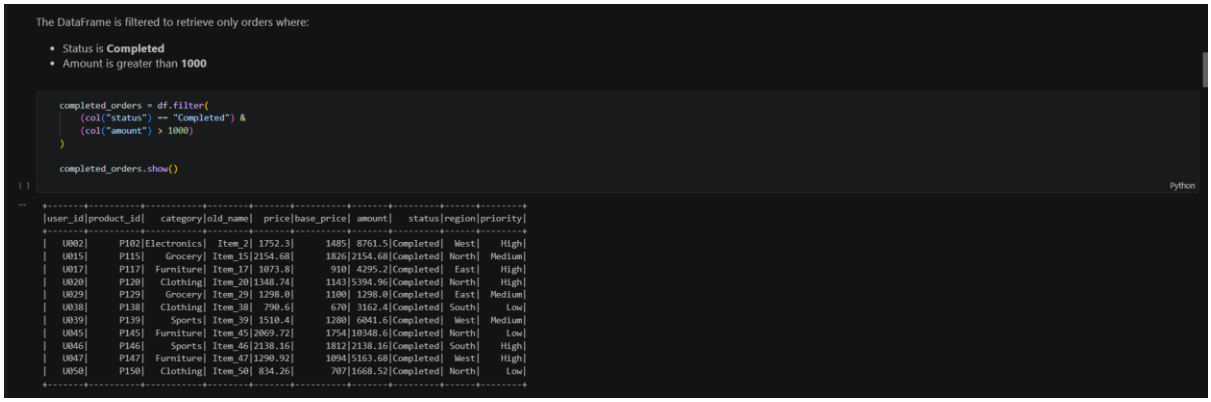
Therefore, the Lineage Graph plays a crucial role in Spark's fault-tolerant architecture.

Q8: Filter all orders where status = "Completed" and amount > 1000.

Answer -

The following Spark code filters the dataset to retrieve only the orders where the **status** is **Completed** and the **amount** is greater than **1000**.

Screenshot:



Explanation

- filter() is used to apply multiple filtering conditions.
- The logical **AND (&)** operator combines both conditions.
- Only records satisfying both conditions are returned.
- show() displays the filtered DataFrame.

Q9: What is Predicate Pushdown in Spark, and why does it improve query performance?

Answer -

Predicate Pushdown

Predicate Pushdown is an optimization technique used by Spark while reading data from storage formats such as **Parquet** and **ORC**. Instead of loading the entire dataset into memory, Spark pushes the filtering conditions down to the storage layer.

Benefits

- Reads only the required data blocks.
- Reduces disk I/O operations.
- Improves query execution time.
- Decreases memory usage.
- Enhances overall Spark performance.

Example

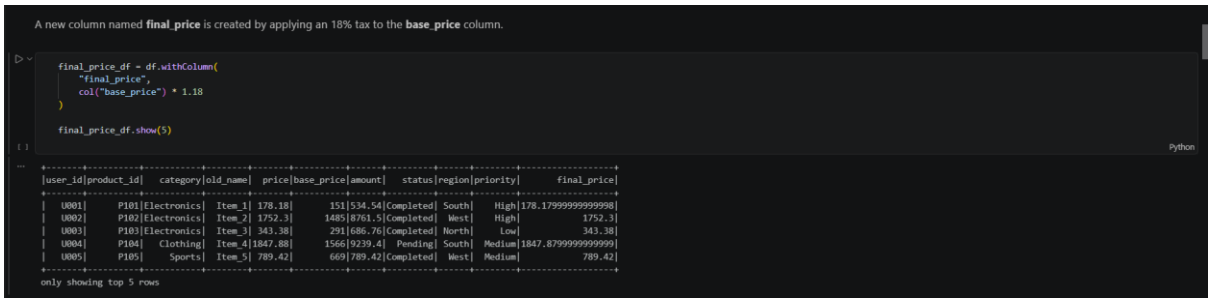
If the query requests only records where:
category = "Electronics"
Spark reads only the relevant data blocks instead of scanning the complete dataset.
This optimization is one of the primary reasons why columnar formats such as **Parquet** perform significantly better than CSV for analytical workloads.

Q10: Add a new column called final_price by applying an 18% tax on the base_price column.

Answer -

The following Spark code creates a new column named **final_price** by applying an **18% tax** to the values in the **base_price** column.

Screenshot:



Explanation

- withColumn() is used to create a new column in the DataFrame.
- The new column **final_price** is calculated by multiplying the **base_price** column by **1.18**.
- show(5) displays the updated DataFrame with the newly created column.

Q11: Differentiate between Transformations and Actions in Spark with suitable examples.

Answer -

Spark operations are broadly classified into **Transformations** and **Actions**.

Transformations

Transformations create a new DataFrame or RDD from an existing one without executing immediately. They are **lazy** in nature and are recorded in the **Lineage Graph (DAG)**.

Examples: select(), filter(), withColumn(), drop(), groupBy()

Actions

Actions trigger the execution of all previously defined transformations and produce the final output.

Examples: show(), count(), collect(), first(), write()

Example

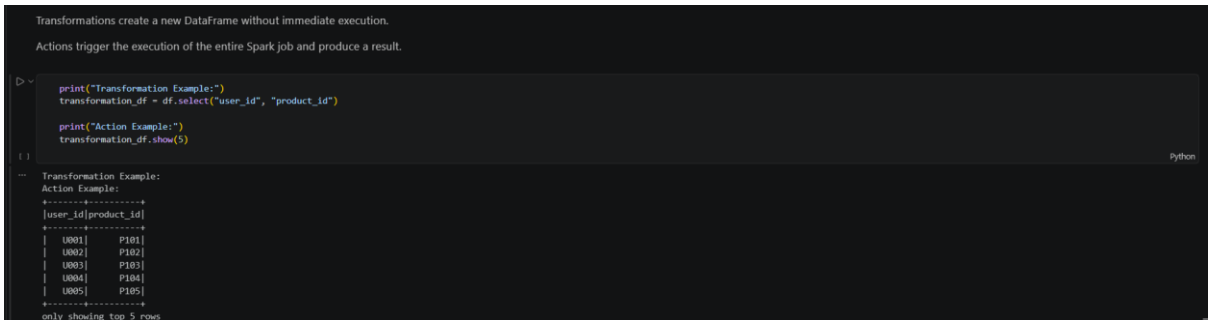
transformation_df = df.select("user_id", "product_id")

transformation_df.show()

In the above example:

- select() is a **Transformation**.
- show() is an **Action**.

Screenshot:



Q12: Build a Spark pipeline that reads a CSV file, converts it to Parquet, reads the Parquet file, filters out rows where user_id is null, and writes the result into a new CSV file.

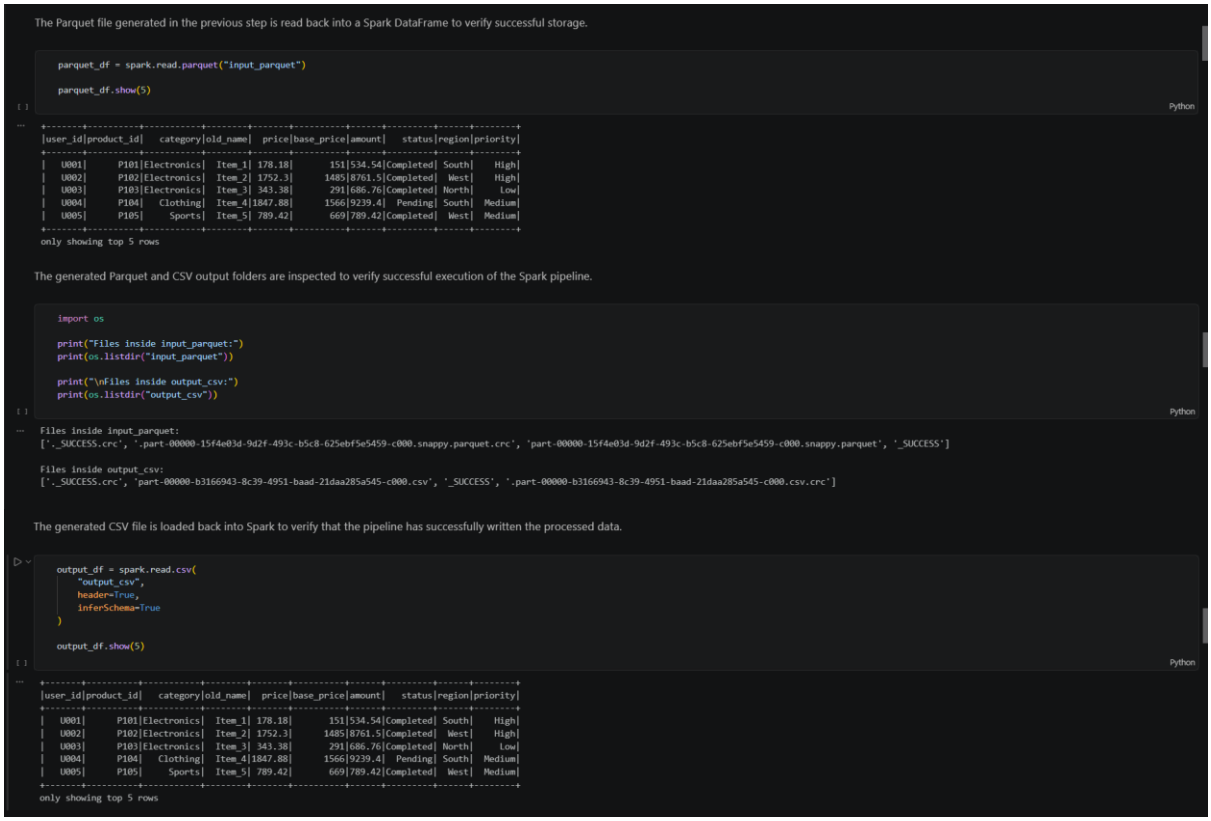
Answer -

The following Spark pipeline demonstrates a complete data processing workflow.

Pipeline Steps

1. Read the CSV file.
2. Convert the DataFrame to Parquet format.
3. Read the generated Parquet file.
4. Filter out rows where user_id is null.
5. Write the processed DataFrame to a new CSV file.
6. Verify the generated output files.

Screenshot:



Explanation

- The original CSV dataset is converted into **Parquet** format.
- The Parquet file is read back into Spark.
- Records containing null values in the **user_id** column are removed.
- The cleaned DataFrame is written into a new CSV file.
- The generated Parquet and CSV folders are verified to ensure successful execution of the Spark pipeline.

Q13: Differentiate between Client Mode and Cluster Mode in Spark Architecture.

Answer -

Client Mode

In **Client Mode**, the Driver program runs on the client machine from which the Spark application is submitted. The Executors run on the worker nodes of the cluster. Since the Driver is outside the cluster, the client machine must remain active throughout the execution of the application.

Cluster Mode

In **Cluster Mode**, the Driver program runs inside the cluster on one of the worker nodes. The Cluster Manager is responsible for launching both the Driver and the Executors. The client can disconnect after submitting the application because the cluster manages the execution independently.

Difference Between Client Mode and Cluster Mode

Client Mode	Cluster Mode
Driver runs on the client machine	Driver runs inside the cluster
Client machine must remain active	Client can disconnect after job submission
Suitable for development and testing	Suitable for production environments
Dependent on client availability	Managed entirely by the cluster

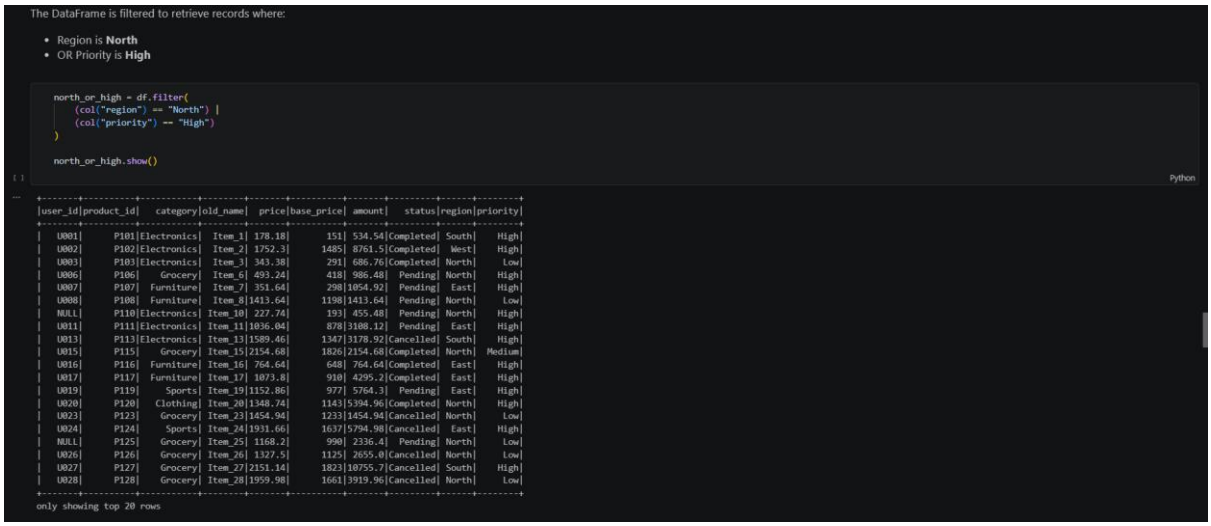
---

Q14: Write a query to filter a dataset for rows where the region is "North" OR the priority is "High".

Answer -

The following query filters the DataFrame to retrieve all records where the **region** is **North** or the **priority** is **High**.

Screenshot:



Explanation

- filter() is used to apply conditions to the DataFrame.
- col("region") == "North" selects records belonging to the North region.
- col("priority") == "High" selects records with High priority.
- The | (**OR**) operator ensures that records satisfying either condition are returned.
- show() displays the filtered results.

Q15: When exploring a dataset, why is it safer to use .show(5) instead of .collect() on a multi-terabyte dataset?

Answer -

.show(5)

The .show(5) function displays only the first five rows of a DataFrame. It retrieves a small subset of data for inspection, making it memory-efficient and suitable for exploring large datasets.

.collect()

The .collect() function retrieves **all** the records from the executors and transfers them to the Driver program as a local collection. For very large datasets, this can consume a significant amount of memory and may cause the Driver to crash with an **Out of Memory (OOM)** error.

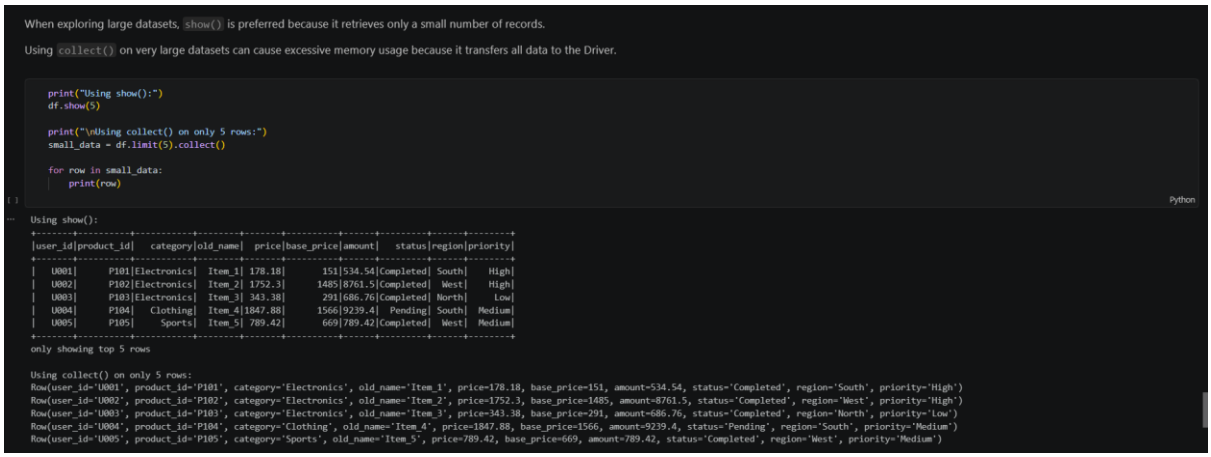
Why .show(5) is Safer

- Retrieves only a few rows instead of the entire dataset.
- Consumes very little memory on the Driver.
- Prevents Out of Memory (OOM) errors.
- Faster for inspecting data.
- Recommended for exploring large or multi-terabyte datasets.

Difference Between .show(5) and .collect()

.show(5)	.collect()
Displays only the first five rows	Retrieves all rows
Memory efficient	Memory intensive
Suitable for large datasets	Suitable only for small datasets

Screenshot:



Conclusion

Apache Spark provides a highly efficient distributed computing framework for processing large-scale datasets. Through this assignment, Spark Architecture, Lazy Evaluation, DAG (Lineage Graph), DataFrame transformations, filtering, schema handling, null handling, and file format optimization using **CSV** and **Parquet** were successfully implemented.

The practical implementation demonstrated how Spark optimizes execution using lazy evaluation, minimizes disk I/O through columnar storage, and efficiently processes data using DataFrame APIs. The assignment also highlighted Spark best practices such as using show() instead of collect() for exploring large datasets and utilizing Parquet for improved analytical performance.

Overall, this assignment provided hands-on experience in building an end-to-end Spark data processing pipeline involving reading data, applying transformations, filtering records, handling null values, and writing processed data into optimized storage formats.